

附件 3：敏捷開發佐證說明書

本次參與「2026 第 31 屆大專校院資訊應用服務創新競賽」之專題「FocusFlow AI」，開發過程以看板（Kanban）作為整體任務追蹤方式，並依需求、測試結果與實際開發狀況持續調整工作內容；其中特定技術模組另採短週期迭代方式進行實作與檢視。團隊透過 Notion、GitHub、會議紀錄與版本控制保留任務、決策及實作歷程，呈現持續學習、回應變化、逐步交付與團隊協作的敏捷精神。

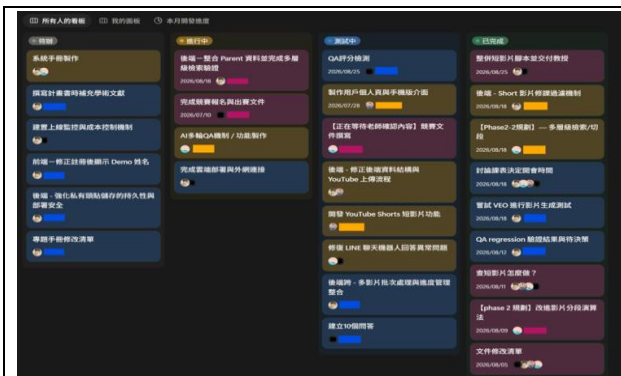
一、敏捷方法與團隊運作

產品目標與待辦事項管理：團隊將需求拆分為身分驗證、語音轉文字、Embedding 與語意檢索、Web 問答、LINE Bot、資料庫、部署、測試與文件等項目，並以 Notion 看板追蹤「待辦、進行中、測試中、已完成」狀態。

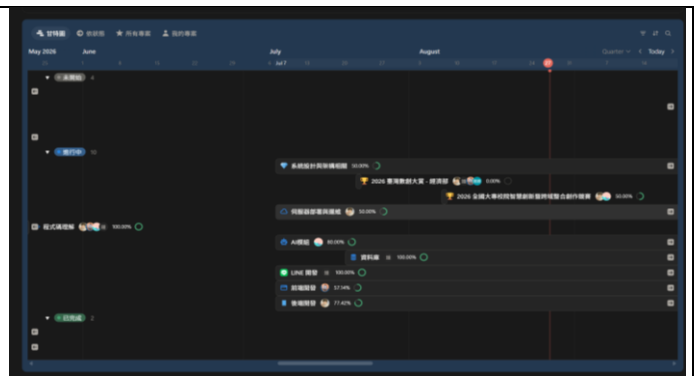
看板與短週期迭代：整體主要使用 Kanban 管理任務，而非全組統一採固定週期 Scrum Sprint；團隊依功能狀態、測試結果與技術問題調整優先順序，需反覆驗證的模組則以短週期完成可執行版本、檢視結果後再修正。

MVP 與範圍調整：優先完成「音訊抽取 → 語音轉文字 → segment 切分 → embedding → 語意檢索 → AI 回答 → timestamp 回溯」文字型端到端流程；音訊與影片多模態 embedding 保留為後續驗證支線。

里程碑管理：以 Milestone 記錄主要章節與功能的預計及實際完成日期；Kanban 管日常工作流，Milestone 掌握階段性期限。



【圖 1】Notion Kanban 任務看板



【圖 2】FocusFlow AI Milestone/Timeline

【圖 1】2026/09/01 Notion 任務紀錄：學生試用版後端 Phase 1 完成跨課程隔離與測試安全邊界，Phase 2 完成 12+2 題 baseline 初跑，並保留人工複核與後續調整項目。

【圖 2】FocusFlow AI Milestone/Timeline：記錄由 MVP、AI Pipeline、學生試用規格、課程隔離到問答 baseline 的迭代順序；時程調整依會議決策、測試結果與任務狀態修正。

二、團隊分工與協作

FocusFlow AI 依成員主要專長分配核心工作，但不採「一人只負責一項工作」的固定分工，並依實際需要跨模組支援。

成員	主要負責	協作內容
組員一	前端與 UI/UX	首頁、登入、Dashboard、角色頁面及前後端介面串接
組員二	後端、API、資料	向量搜尋、答案生成、MongoDB 資料模型與索引
組員三	AI 技術	Gemini Embedding、STT、逐字稿、片段處理、RAG/語意檢索、文件
組員四	部署與文件	GitHub Actions、會議紀錄、README、Swagger、架構文件

四位成員亦共同支援資料庫欄位設計、API 與前端串接、系統測試、技術問題排查及章節文件確認。

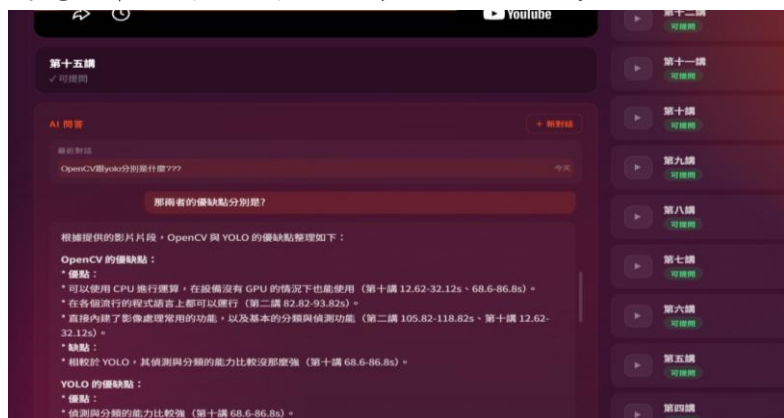
三、完成定義與驗證方式

功能完成不以「程式碼寫完」作為唯一判準，而需確認 API、前後端串接、MongoDB 資料、AI Pipeline、問答內容、LINE 流程、timestamp/YouTube 跳轉、課程權限與錯誤處理是否符合預期。

驗證面向	怎麼判斷做完了
------	---------

功能與串接	主要流程走得通，錯誤狀況也有處理，測試結果能重跑。
問答與回溯	逐題檢查答案、來源、時間點與影片跳轉，API 成功不代表答案正確。
權限與拒答	學生只能查有權限的課；教材不夠時要確實拒答，說法與來源要一致。
自動化測試	三端都有測試：批次功能 447、168、14 項通過，課程隔離另有 514 項。
外部服務	Atlas、Gemini、LINE、YouTube 都連真實服務驗證，不用 mock 代替。

團隊將「程式實作完成」「自動化測試通過」「外部服務可用」與「正式驗收通過」視為不同狀態，避免將單一測試結果誤認為整體功能完成。



【圖 3】課程多輪問答與影片時間戳回溯功能驗證

【圖 3】2026/09/01 課程問答 baseline：以 OpenCV 正式 15 支影片執行 12 題正向問題與 2 題負向問題，確認測試過程未引用其他課程，資料庫寫入為 0；N01 跨課程隔離成功，N02 拒答 citation 仍需改善。

四、開發、檢視與調整紀錄

時間與事件	發現了什麼	怎麼調整	證據
2026/03 ~ 04 縮小範圍	架構圖塞了短影音、推播與儀表板，做不完也驗不完。	先做登入、課程、影片、問答四件事，上傳與轉檔改為全自動。	Notion 保存六次方向調整；04/21 進度顯示主線與測試完成。
2026/08/04 換模型	Google 宣布舊模型 08/10 起停用，不換會讓向量對不上。	三端同步換成穩定版，新檢索先用開關關掉。	Phase2-2 報告保存決策與驗收標準，離線驗證通過。
2026/08/12 ~ 08/14 批次處理	影片一支一支處理，容易失敗、重複排程或狀態遺失。	改成批次任務，加上狀態、部分成功、重試與續跑。	Notion 記錄 447、168、14 項測試通過，API 文件同步更新。
2026/08/30 ~ 09/01 課程隔離	只靠課程欄位過濾，還是可能引到別的課。	改用正式影片白名單，名單為空就直接拒答。	Phase1 完成隔離，後端回歸測試 514 項全數通過。
2026/09/01 問答測試	負向題會拒答，但來源顯示不一致；兩題超出教材深度。	先分清是程式、題目還是搜尋品質問題，不放寬檢索門檻。	Phase 2 完成 12+2 題題庫與第一次量測，全部跑完且沒引到別的課。

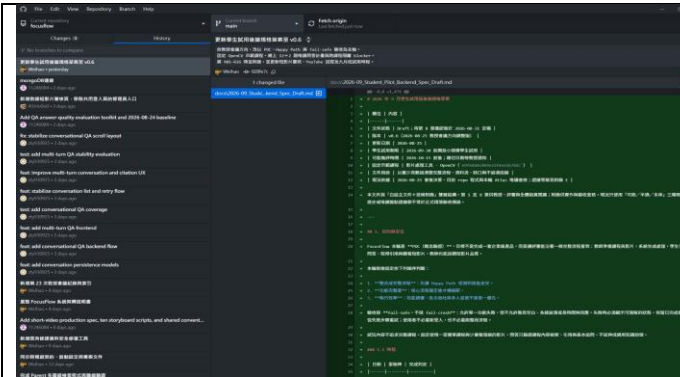
上述調整分別由範圍檢視、教授建議、外部模型風險與測試結果觸發，再回到 Notion 任務重新拆分、實作與驗證。團隊不把**功能可執行**當成唯一終點，而是保留問題、決策、驗證結果與下一輪待辦，使優先順序能依證據調整。

五、版本控制、部署與文件維護

項目	目前作法	說明
自動化測試	Backend、Frontend、AI Pipeline 測試	功能修改後執行對應測試；重要改版再執行完整回歸測試並保存結果。

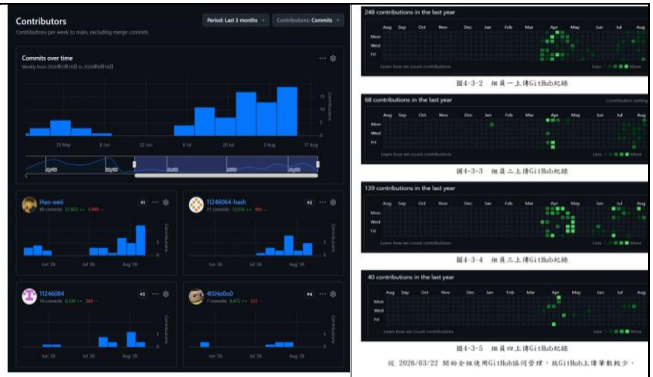
持續部署 (CD)	GitHub Actions Self-Hosted Runner	main 更新後自動取得程式碼、安裝依賴、建置前端、重啟 Backend 並重新載入 Nginx。
-----------	-----------------------------------	---

目前流程定位為「本機自動化測試+GitHub Actions 持續部署」，尚未宣稱為完整 CI/CD。多影片批次任務紀錄中也明確標示，push 到 main 會部署目前工作區多項變更，但 workflow 仍沒有自動測試與部署後 Health Check；因此 CI gate、Health Check 與自動回滾列為下一階段 DevOps 待辦事項。



【圖 4】GitHub Commit／版本變更

【圖 4】2026/08/14 多影片批次處理驗證：Notion 任務記錄 Backend 447/447、AI Pipeline 168/168、Frontend 14/14 測試通過，並完成 lint、build 與 OpenAPI 文件同步；正式 STT/Gemini 與壓力測試尚未列入通過。



【圖 5】全組 commits 與成員 Contributions

【圖 5】團隊 Contributions 或 task/commit/test 對照：僅用來呈現成員參與與版本變更軌跡；功能品質仍以 Notion 任務、commit 內容、測試結果與 baseline 紀錄判定，不以 contribution 數字單獨代表完成度。

六、可稽核佐證

FocusFlow AI 的敏捷開發歷程由多種工具共同形成可追溯佐證，不同資料分別對應任務狀態、技術決策、版本變更、部署歷程與系統設計。

佐證鏈	可稽核方式
問題與待辦	Notion 任務：問題、負責人、狀態與測試結果。
決策與分工	會議紀錄：教授建議、調整原因與負責模組。
規格檢視	試用版建議頁：組員意見與逐條回覆。
實作變更	commit：對應的程式、文件與測試改動。
測試結果	測試紀錄：執行日期、範圍與通過數。
部署結果	GitHub Actions：main 的部署過程。
文件契約	Swagger 與系統手冊：API 與功能邊界。

七、敏捷開發實踐總結

評分面向	我們實際怎麼做
適應性	每次週會、規格檢視或測試結果出來就重排待辦，沒解決的留到下一輪。
可視性	Notion、會議紀錄、commit、測試與 Milestone 分別留下問題、決策、實作與驗證。
技術實踐	三端都有自動化測試，main 由 GitHub Actions 自動部署；健康檢查還在補。
逐步交付	先做完影片知識化、課程問答與時間點回溯，再加批次、續跑與課程隔離。
團隊協作	各人負責一個模組，靠 API 文件、測試與會議紀錄對齊。

FocusFlow AI 以短週期迭代推進開發：每週與教授檢視進度並決定下一輪範圍（迭代規劃），工作在 Notion 看板拆成小任務並公開狀態（任務可視化），每一輪都交付可真實操作的功能而不是只交文件（增量交付），功能通過測試後合併並自動部署（持續整合與部署）；檢視發現的問題不會被跳過，而是回到看板重新排序成為下一輪待辦（依回饋調整）。